

กลุ่ม [10] — [Collection & String]

1. [นางสาว ชัญญา เกระสวัสด์] — Concept + Short Code Illustration
2. [นาย ภัทรกฤต สังครุ] — Detailed Code + Live Demo
3. [นาย ศุภณัฐ วิริยะบรอนันต์] — Rust vs Other Language + PPL Analysis
4. [นายสิริวิชญ์ ปิ่นแสง] — Exercises + Common Mistakes + Challenge

Agenda — โครงสร้างการนำเสนอ (รวม ~20 นาที)

แต่ละคนนำเสนอประมาณ 5 นาที ต่อเนื่องกันตามลำดับ

1

Member 1- นางสาว ชัญญา เระสวัสดิ์

**Concept + Short Code
Illustration**

แนวคิดหลักของหัวข้อ พร้อมโค้ดตัวอย่างสั้น ๆ

~5 นาที

2

Member 2 - นายภัทรกฤต สังครบ

Detailed Code + Live Demo

อธิบายโค้ดเชิงลึกและสาธิตการทำงานจริง

~5 นาที

3

Member 3 - นาย ศุภณัฐ วิริยะนรอนันต์

**Rust vs Other Language +
PPL Analysis**

เปรียบเทียบภาษา และวิเคราะห์เชิง
Programming Language

~5 นาที

4

Member 4 : สิริวิชญ์ ปิ่นแสง

**Exercises + Common
Mistakes + Challenge**

ข้อผิดพลาดที่พบบ่อย และโจทย์ท้าทายผู้ฟัง

~5 นาที

[Collection&String] คืออะไร? (Concept)

- Collection & String คือ โครงสร้างข้อมูลที่ใช้สำหรับจัดเก็บข้อมูลหลายค่าไว้ภายในโครงสร้างเดียว ทำให้สามารถรวบรวมและจัดการข้อมูลที่เกี่ยวข้องกันได้อย่างเป็นระบบ
- ข้อมูลที่จัดเก็บใน Collection สามารถมีจำนวนมาก และในบางประเภทสามารถ เพิ่มหรือลดข้อมูลระหว่างการทำงานของโปรแกรมได้
- **ความสำคัญของ Collection**
ในการเขียนโปรแกรม ข้อมูลมักไม่ได้มีเพียงค่าเดียว เช่น ข้อมูลนักเรียน รายการสินค้า หรือข้อมูลจำนวนมาก การมี Collection ช่วยให้เราสามารถ จัดเก็บและจัดการข้อมูลหลายค่าได้สะดวกกว่าการสร้างตัวแปรแยกกัน

[Array] คืออะไร? (Concept)

- **Array** คือ โครงสร้างข้อมูลที่ใช้เก็บค่าหลายค่าในตัวแปรเดียว
โดย -ข้อมูลเป็น ชนิดเดียวกัน
-มี **ขนาดคงที่** ไม่สามารถเพิ่มหรือลดจำนวนสมาชิกได้
-เข้าถึงข้อมูลด้วย **Index[]** (เริ่มที่ 0)

หากต้องการเปลี่ยนค่าภายใน Array สามารถใช้ **mut** ได้

- **[เหตุผลที่ Rust ออกแบบแนวคิดนี้]**
 - เก็บข้อมูลหลายค่าไว้ในโครงสร้างเดียว
 - กำหนดขนาดแน่นอน → เหมาะกับข้อมูลที่รู้จำนวนล่วงหน้า
 - มีการตรวจสอบขอบเขตการเข้าถึงข้อมูล
- **[ปัญหาที่แนวคิดนี้ช่วยแก้ไข]**
 - ลดการสร้างตัวแปรหลายตัวสำหรับข้อมูลประเภทเดียวกัน
 - จัดการข้อมูลจำนวนมากได้ง่ายขึ้น

ชนิดข้อมูลของสมาชิก

จำนวนสมาชิก

```
fn main() {
    let scores: [i32; 3] = [80, 90, 75];

    println!("{}", scores[0]);
    println!("{}", scores[1]);
    println!("{}", scores[2]);
}
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

ตำแหน่งสำหรับนำค่าของ text มาแสดง

80
90
75

Array เก็บข้อมูลหลายค่าชนิดเดียวกัน

[Tuple] คืออะไร? (Concept)

- **Tuple** คือ โครงสร้างข้อมูลที่ใช้เก็บค่าหลายค่าในตัวแปรเดียว
โดย -ข้อมูล สามารถเป็นคนละชนิดกัน
-มี **ขนาดคงที่** ไม่สามารถเพิ่มหรือลดจำนวนสมาชิกได้
-เข้าถึงข้อมูลด้วย **Index** (indexแรกเริ่มที่ 0)]

เข้าถึงสมาชิก ใช้ **.index**

- **[เหตุผลที่ Rust ออกแบบแนวคิดนี้]**
 - เก็บข้อมูลที่ **เกี่ยวข้องกันแต่มีชนิดต่างกัน** ไว้ด้วยกัน
 - ลดการสร้างตัวแปรหลายตัวสำหรับข้อมูลชุดเดียวกัน
 - กำหนดจำนวนและชนิดของข้อมูลได้ชัดเจน
- **[ปัญหาที่แนวคิดนี้ช่วยแก้ไข]**
 - ลดปัญหาการต้องสร้างตัวแปรแยกสำหรับข้อมูลที่เกี่ยวข้องกัน
 - ช่วยจัดกลุ่มข้อมูลที่ มีชนิดข้อมูลแตกต่างกัน ให้อยู่ในตัวแปรเดียว

```
fn main() {
    let student = ("Alice", 20, true);

    println!("{}", student.0);
    println!("{}", student.1);
    println!("{}", student.2);
}
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
"Alice" //→ String literal
20      //→ จำนวนเต็ม
true    //→ Boolean
```

Tuple เก็บข้อมูลหลายค่าต่างชนิดกันได้

[Vector] คืออะไร? (Concept)

- Vector หรือ Vec<T>
คือ โครงสร้างข้อมูลที่ใช้เก็บค่าหลายค่าในตัวแปรเดียว
โดย -ข้อมูลทุกตัวต้องเป็นชนิดเดียวกัน
-ข้อมูลสามารถเพิ่มหรือลดจำนวนสมาชิกได้
-เข้าถึงข้อมูลด้วย Index (indexแรกเริ่มที่ 0)]
- [เหตุผลที่ Rust ออกแบบแนวคิดนี้]
-ใช้เก็บข้อมูลที่ จำนวนไม่แน่นอน
-สามารถ เพิ่ม / ลบข้อมูล ระหว่างโปรแกรมทำงาน
-ช่วยจัดการข้อมูลหลายค่าไว้ในโครงสร้างเดียว
- [ปัญหาที่แนวคิดนี้ช่วยแก้ไข]
-Array ขนาดคงที่ เช่น [80, 90, 75]
ถ้าต้องการเพิ่มข้อมูล จะไม่สามารถเพิ่มเข้า Array เดิมได้
-Vector ขนาดยืดหยุ่น เช่น [80, 90, 75] → [80, 90, 75, 85]
จึงเหมาะกับข้อมูลที่ จำนวนเปลี่ยนแปลงระหว่างการทำงาน

ยังมี ใช้ `len()` เพื่อดูจำนวนข้อมูล รวมถึงใช้ `pop()` เพื่อลบข้อมูลตัวสุดท้ายได้

```
fn main() {
    let mut scores = vec![80, 90, 75];

    scores.push(85);    //การเพิ่ม

    println!("{:?}", scores);
}
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
[80, 90, 75, 85]
```

Vector เหมือน array แต่เพิ่มลดสมาชิกได้

[String] คืออะไร? (Concept)

- String คือ โครงสร้างข้อมูลที่ใช้เก็บและจัดการข้อความ โดย
 - ข้อมูลภายใน String เป็น ชุดของตัวอักษร (UTF-8)
 - สามารถ เพิ่ม,ลด,แก้ไข จำนวนสมาชิกได้
 - ขนาดของ String สามารถเปลี่ยนแปลงได้

push() → เพิ่ม ตัวอักษร 1 ตัว (char)

push_str() → เพิ่ม ข้อความ (&str)

- [เหตุผลที่ Rust ออกแบบแนวคิดนี้]
 - ใช้จัดการ ข้อความที่มีขนาดไม่แน่นอน
 - รองรับข้อความหลายภาษาและ UTF-8
 - สามารถปรับขนาดข้อความตามการใช้งาน

- [ปัญหาที่แนวคิดนี้ช่วยแก้ไข]
 - การจัดการข้อความที่มีความยาวเปลี่ยนแปลง เช่น

"Hello" → "Hello Rust" ไม่ต้องกำหนดขนาดข้อความตายตัวตั้งแต่เริ่มต้น

- String ไม่สามารถเข้าถึงข้อมูลด้วย Index เช่น text[0] ได้โดยตรง เพราะตัวอักษรใน UTF-8 อาจใช้จำนวนไบต์แตกต่างกัน

```
fn main() {
    let mut message = String::from("Hello");

    message.push_str(" Rust");

    println!("{}", message);
}
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
Hello Rust
```

Stringเหมาะกับข้อความที่แก้ไขได้
ระหว่างโปรแกรมทำงาน

[&str] คืออะไร? (Concept)

- &str คือ String Slice ที่ใช้สำหรับอ้างอิงข้อมูลข้อความ
 - เป็นการอ้างอิงข้อมูล String ที่มีอยู่แล้ว โดย ไม่ได้เป็นเจ้าของข้อมูล
 - ข้อมูลที่อ้างอิงเป็นข้อความที่เข้ารหัสแบบ UTF-8
 - สามารถอ้างอิงข้อความบางส่วนของ String ได้
- [เหตุผลที่ Rust ออกแบบแนวคิดนี้]
 - ใช้สำหรับเข้าถึงหรืออ้างอิงข้อความโดยไม่ต้องเป็นเจ้าของข้อมูล
 - ลดการสร้างหรือคัดลอกข้อมูล String ที่ไม่จำเป็น
 - สามารถใช้รับข้อความได้ทั้งจาก String และ String Literal
- [ปัญหาที่แนวคิดนี้ช่วยแก้ไข]
 - การส่งหรือใช้งานข้อความโดยไม่จำเป็นต้องสร้าง String ใหม่

```
fn main() {
    let message = String::from("Hello Rust");
    let text: &str = &message;

    println!("{}", text);
}
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
Hello Rust
```

Example: ระบบจัดการสต็อกร้านสะดวกซื้อ

Part: Struct + Trait

```
use std::collections::HashMap;
use std::fmt;
```

```
#[derive(Debug, Clone)]
struct Product {
    name: String,
    category: String,
    stock: u32,
    reorder_level: u32,
}
```

```
impl fmt::Display for Product {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(
            f,
            "{:<15} ({:<10}) คงเหลือ {} ชิ้น",
            self.name, self.category, self.stock
        )
    }
}
```

#[derive(Debug, Clone)] → เพิ่มความสามารถให้ Struct

derive คือการให้ Rust สร้าง implementation ของ trait บางอย่างให้เราอัตโนมัติ ในที่นี้มี Debug สำหรับ debug/แสดงข้อมูล และ Clone สำหรับสร้างสำเนาของข้อมูล

struct Product → โครงสร้างข้อมูลสินค้า

ตรงนี้เราสร้าง Struct ชื่อ Product เพื่อเก็บข้อมูลของสินค้า 1 ชิ้น โดยมีชื่อสินค้า หมวดหมู่ จำนวนคงเหลือ และจำนวนขั้นต่ำที่ควรสั่งเพิ่ม

impl Display → กำหนดวิธีแสดงผลของ Product

ตรงนี้คือการ implement Display ให้กับ Product เพื่อกำหนดว่าเวลาเราเอา Product ไป print จะแสดงผลออกมาในรูปแบบไหน

Example: ระบบจัดการสต็อกร้านสะดวกซื้อ

Part: Functions

```
fn stock_status(stock: u32, reorder_level: u32) -> &'static str {
    match stock {
        0 => "หมดสต็อก",
        s if s <= reorder_level => "ใกล้หมด ต้องสั่งเพิ่ม",
        _ => "ปกติ",
    }
}

fn split_name_category<'a>(name: &'a str, category: &'a str) -> (&'a str, &'a str) {
    (name, category)
}
```

fn stock_status() → Match Guard

เป็นฟังก์ชันที่รับค่า stock กับ reorder_level แล้วคืนข้อความบอกสถานะของสินค้า

ตรงนี่คือ Match Guard ใช้ if เพิ่มเงื่อนไขให้กับ pattern ถ้า stock น้อยกว่าหรือเท่ากับ reorder_level ก็จะแสดงว่า ใกล้หมด ต้องสั่งเพิ่ม

fn split_name_category() → Tuple + Lifetime

Function นี้รับ name กับ category ที่เป็น &str แล้วคืนค่ากลับมาเป็น Tuple ที่มี 2 ค่า

'a เป็น Lifetime ที่บอกว่า reference ที่รับเข้ามา และ reference ที่คืนออกไปมีช่วงอายุการใช้งานที่สัมพันธ์กัน

(name, category)

ตรงนี่คือการคืนค่าแบบ Tuple ทำให้เราสามารถคืน name และ category กลับมาพร้อมกันได้

Example: ระบบจัดการสต็อกร้านสะดวกซื้อ

Part: Main

```
fn main() {
    let allowed_categories: [&str; 4] = ["เครื่องดื่ม", "ขนม", "ของใช้", "อาหารแห้ง"];
    println!("=== Array Demo ===");
    println!("หมวดหมู่ที่รับ: {}\n", allowed_categories.join(", "));

    let raw_stock: &str = "\
น้ำดื่ม,เครื่องดื่ม,120,30
มาล่า,อาหารแห้ง,8,20
ทิชชู,ของใช้,45,15
ขนมปัง,ขนม,0,10
กาแฟกระป๋อง,เครื่องดื่ม,60,25
สบู่,ของใช้,5,10
ปลากระป๋อง,อาหารแห้ง,300,40";

    let mut inventory: Vec<Product> = Vec::new();
```

let allowed_categories → Array: เก็บหมวดหมู่สินค้า

let raw_stock → &str: เก็บข้อมูลสต็อกดิบเป็นข้อความ

let mut inventory → Vec: สร้างพื้นที่ไว้เก็บ Product หลากๆ ตัว
ได้ข้อมูลจาก raw_stock ที่ผ่านการตรวจสอบจะถูกใส่เข้ามาตรงนี้

Array = กำหนดหมวด → &str = รับข้อมูลดิบ → Vec = เตรียมเก็บสินค้า

Example: ระบบจัดการสต็อกร้านสะดวกซื้อ

Part: Main (ต่อ)

```

for line in raw_stock.lines() {
    let fields: Vec<&str> = line.split(',').collect();
    if fields.len() != 4 {
        continue;
    }

    let (name, category) = split_name_category(fields[0].trim(), fields[1].trim());

    if !allowed_categories.contains(&category) {
        eprintln!("ข้ามบรรทัด (หมวดหมู่ไม่รู้จัก): {}", line);
        continue;
    }

    let stock: u32 = match fields[2].trim().parse() {
        Ok(v) => v,
        Err(_) => {
            eprintln!("ข้ามบรรทัด (จำนวนสต็อกไม่ใช่ตัวเลข): {}", line);
            continue;
        }
    };

    let reorder_level: u32 = match fields[3].trim().parse() {
        Ok(v) => v,
        Err(_) => {
            eprintln!("ข้ามบรรทัด (reorder level ไม่ใช่ตัวเลข): {}", line);
            continue;
        }
    };

    inventory.push(Product {
        name: name.to_string(), // &str -> String เพราะต้องเป็นเจ้าของข้อมูลเอง
        category: category.to_string(),
        stock,
        reorder_level,
    });
}

```

let fields: Vec<&str> → แยกข้อมูลแต่ละบรรทัดด้วย ,
เช่น น้ำดื่ม,เครื่องดื่ม,120,30 → 4 ช่อง

if fields.len() != 4 { ... } ถ้าข้อมูลไม่ครบ 4 ช่อง → ข้ามบรรทัดนั้น

if !allowed_categories.contains(&category) { ... }
เช็คหมวดหมู่อยู่ในรายการที่ร้านอนุญาตไหม

let stock → แยกข้อมูลแต่ละบรรทัดด้วย ,
แปลงจำนวนสต็อกจากข้อความเป็น u32
Ok = แปลงสำเร็จ, **Err** = แปลงไม่ได้ → ข้ามข้อมูล

let reorder_level → จุดที่กำหนดว่า สต็อกเหลือเท่าไรแล้วควรสั่งเพิ่ม
เช่น reorder_level = 20 ก็คือ เหลือ 20 หรือน้อยกว่า = ต้องสั่งเพิ่ม

inventory.push → เก็บข้อมูลลง Vec
เมื่อข้อมูลผ่านการตรวจสอบแล้ว → สร้าง Product แล้วเก็บเข้า inventory

Concept: แยกข้อมูล → ตรวจสอบ → แปลงข้อมูล → เก็บลง Vec

Example: ระบบจัดการสต็อกร้านสะดวกซื้อ

Part: Main (ต่อ)

```

let mut stock_by_category: HashMap<String, u32> = HashMap::new();
for p in &inventory {
    *stock_by_category.entry(p.category.clone()).or_insert(0) += p.stock;
}

println!("\n=== รวมสต็อกตามหมวดหมู่ ===");
for cat in allowed_categories.iter() {
    let total = stock_by_category.get(*cat).copied().unwrap_or(0);
    println!("{:<12} | รวม {} ชิ้น", cat, total);
}

println!("\n=== สถานะสต็อกรายสินค้า ===");
for p in &inventory {
    println!("{}", p, stock_status(p.stock, p.reorder_level));
}

let mut urgency_ranking = inventory.clone();
urgency_ranking.sort_unstable_by(|a, b| a.stock.cmp(&b.stock)); // closure เปรียบสต็อก น้อย -> มาก

println!("\n=== อันดับสินค้าที่ต้องสั่งด่วนที่สุด ===");
for (i, p) in urgency_ranking.iter().take(3).enumerate() {
    println!("{}", i, p.name, p.stock);
}

let need_restock: Vec<String> = inventory
    .iter()
    .filter(|p| p.stock <= p.reorder_level)
    .map(|p| p.name.clone())
    .collect();

println!("\nสินค้าที่ต้องสั่งเพิ่ม: {}", need_restock.join(", "));

```

let mut stock_by_category → HashMap เก็บ หมวดหมู่ → จำนวนสต็อกรวม

รวมยอดสต็อก วนดูสินค้าทีละตัว แล้วเอาสต็อกไป **บวกสะสม**ตามหมวดหมู่

for cat in allowed_categories.iter() → **วนทีละหมวด** แล้วดึงยอดสต็อกรวมจาก HashMap มาแสดง (ถ้าไม่มีข้อมูล → ให้เป็น 0)
(ใช้ **.get()** ในการดึงข้อมูล)

for p in &inventory → วนดูสินค้าทุกตัวใน inventory
แสดงข้อมูลสินค้า และเรียก **stock_status()** เพื่อบอกว่า ปกติ / ใกล้หมด / หมด

let mut urgency_ranking → **.Clone** ข้อมูลออกมาก่อน เพื่อเอาไปเรียงโดย
ไม่กระทบ inventory ตัวเดิม
urgency_ranking.**sort_unstable_by** เปรียบ stock แล้วเรียงจาก น้อย → มาก

.take(3).enumerate() → วนข้อมูลที่เรียงไว้ **เอาแค่ 3 ตัวแรก** → พร้อมนับลำดับ
เอาสินค้า 3 ตัวที่เหลือน้อยที่สุดมาแสดง พร้อมอันดับและจำนวนสต็อก

let need_restock → สร้างตัวแปรเก็บ**รายชื่อสินค้าที่ต้องสั่งเพิ่ม** โดยเก็บเป็น Vec<String>

inventory.**iter()** → วนดูสินค้าทั้งหมด
filter() → กรองเฉพาะสินค้าที่ ถึงจุดต้องสั่งเพิ่ม
map() + collect() → เอาเฉพาะ ชื่อสินค้า แล้วรวมเป็น Vec
join → รวมชื่อสินค้าให้เป็นข้อความเดียว คั่นด้วย ,

Rust vs [Other Language]

Aspect	Rust	[C]
Syntax	บังคับ Explicit Intent (mut, &) แยก String/&str ชัดเจน ห้าม Index String (s[0] = Error) ต้องใช้ .chars()	ใช้ char* หรือ char[] และใช้ s[0] เข้าถึง Byte แรกได้ทันที (เสี่ยง Buffer Overflow)
Semantics / Behavior	Move Semantics (โอนเจ้าของ) และ Fail Fast (Panic ทันทีถ้าตัด UTF-8 ผิดขอบเขต)	Copy Pointer (ชี้ Address เดียวกัน) และจัดการ UTF-8 เอง (ถ้าตัดผิดอาจได้ผลลัพธ์ที่พังโดยไม่แจ้งเตือน)
Type System	Static, Strong, No Implicit Coercion, มี Traits และ Generics ที่ปลอดภัย	Weak Typing, ไม่มี Generics ที่ปลอดภัย, แปลง Type ได้ง่ายอย่างอิสระจนเกิดบั๊กได้ง่าย
Memory Management	RAII (No GC): Metadata อยู่ Stack, Data อยู่ Heap คืน Memory ทันทีที่หลุด Scope	Manual Memory: ต้องใช้ malloc() และ free() เอง เสี่ยง Memory Leak และ Double Free สูง
Safety	Fearless Concurrency: Compiler เช็ค Data Race ให้ 100% ปลอดภัยจาก Null Pointer, Buffer Overflow, Dangling Pointer	เสี่ยง Segfault, Data Race และช่องโหว่ด้านความปลอดภัยสูงมาก

Rust vs [Other Language]

Aspect	Rust	[C++]
Syntax	บังคับ Explicit Intent (mut, &) แยก String/&str ชัดเจน ห้าม Index String (s[0] = Error) ต้องใช้ .chars()	ใช้ std::string และ std::string_view (C++17) แต่ไม่มี Built-in Ownership บังคับ
Semantics / Behavior	Move Semantics (โอนเจ้าของ) และ Fail Fast (Panic ทันทีถ้าตัด UTF-8 ผิดขอบเขต)	มี Move Constructors แต่ซับซ้อนและเรียกใช้ไม่สม่ำเสมอ (Complex Semantics)
Type System	Static, Strong, No Implicit Coercion, มี Traits และ Generics ที่ปลอดภัย	มี Templates และ string_view แต่ยังอนุญาต Implicit Casting และไม่บังคับกฎ Ownership
Memory Management	RAII (No GC): Metadata อยู่ Stack, Data อยู่ Heap คืน Memory ทันทีที่หลุด Scope	RAII (Smart Pointers) แต่เป็นทางเลือก (Optional) ยังสามารถใช้ new/delete ได้
Safety	Fearless Concurrency: Compiler เช็ค Data Race ให้ 100% ปลอดภัยจาก Null Pointer, Buffer Overflow, Dangling Pointer	เสี่ยง Dangling Pointer และ Memory Leak ถ้าผู้เขียนไม่ได้ใช้ Smart Pointer อย่างระมัดระวัง

Rust vs [Other Language]

Aspect	Rust	[Java]
Syntax	บังคับ Explicit Intent (mut, &) แยก String/str ชัดเจน ห้าม Index String (s[0] = Error) ต้องใช้ .chars()	ใช้ s.charAt(0) (แต่ Java เก็บ String เป็น UTF-16 ไม่ใช่ UTF-8)
Semantics / Behavior	Move Semantics (โอนเจ้าของ) และ Fail Fast (Panic ทันทีถ้าตัด UTF-8 ผิดขอบเขต)	Copy Reference (ชี้ Object เดียวกัน) และ Silent Failure ถ้าจัดการ Unicode ผิด
Type System	Static, Strong, No Implicit Coercion, มี Traits และ Generics ที่ปลอดภัย	Static แต่มี Type Erasure (ข้อมูล Type หายไปตอน Runtime)
Memory Management	RAII (No GC): Metadata อยู่ Stack, Data อยู่ Heap คืน Memory ทันทีที่หลุด Scope	Garbage Collector: จัดการ Memory อัตโนมัติ แต่มี GC Pause (โปรแกรมหยุดชั่วคราว)
Safety	Fearless Concurrency: Compiler เช็ค Data Race ให้ 100% ปลอดภัยจาก Null Pointer, Buffer Overflow, Dangling Pointer	มี NullPointerException, จัดการ Concurrency ยาก (ต้องใช้ Lock เอง เสี่ยง Deadlock)

Rust vs [Other Language]

Aspect	Rust	[Python]
Syntax	บังคับ Explicit Intent (mut, &) แยก String/&str ชัดเจน ห้าม Index String (s[0] = Error) ต้องใช้ .chars()	ยืดหยุ่น เขียนง่าย แต่ s[0] อาจได้ผลลัพธ์ผิดถ้าเป็น Unicode ที่ซับซ้อน
Semantics / Behavior	Move Semantics (โอนเจ้าของ) และ Fail Fast (Panic ทันทีถ้าตัด UTF-8 ผิดขอบเขต)	Copy Reference (Mutable Object) และ Silent Failure (ได้ค่าผิดโดยไม่ Error)
Type System	Static, Strong, No Implicit Coercion, มี Traits และ Generics ที่ปลอดภัย	Dynamic Typing (Duck Typing) ตรวจสอบตอน Runtime ทำให้เกิด Runtime Error ได้ง่าย
Memory Management	RAII (No GC): Metadata อยู่ Stack, Data อยู่ Heap คืน Memory ทันทีที่หลุด Scope	Garbage Collector: ใช้ Memory สูง และมี Overhead จากการที่ทุกอย่างเป็น Object
Safety	Fearless Concurrency: Compiler เช็ค Data Race ให้ 100% ปลอดภัยจาก Null Pointer, Buffer Overflow, Dangling Pointer	ติด GIL (Global Interpreter Lock) ทำให้ Multi-threading ทำงานได้ไม่เต็มที่

Rust vs [Other Language]

Analysis

Analysis: ความแตกต่างระหว่าง Rust กับ C/C++, Java, และ Python ไม่ได้อยู่ที่ Syntax แต่อยู่ที่ "ปรัชญาการออกแบบ" (Design Philosophy):

- *C / C++ (Control):* ให้อิสระในการควบคุม Memory สูงสุด → เร็วที่สุด แต่แลกมาด้วยความเสี่ยง (*Memory Leak, Segfault, Dangling Pointer*) และไม่มี *Built-in Ownership* บังคับ
- *Java / Python (Convenience):* ใช้ GC จัดการ Memory ให้ → เขียนง่าย แต่แลกมาด้วย *Runtime Overhead* (*GC Pause, Type Erasure, GIL, Dynamic Typing*)
- *Rust (Safety + Performance):* ปฏิเสธการประนีประนอม → ใช้ *Ownership* ตรวจสอบตอน *Compile* ทำให้ได้ทั้งความเร็วและความปลอดภัยโดยไม่มี GC

ตัวอย่างที่ชัดเจน (*String & UTF-8*):

- *C: s[0]* ได้ Byte แรก (เสี่ยง *Buffer Overflow*)
- *Python: s[0]* อาจได้ผลลัพธ์ผิดถ้าเป็น *Unicode* (*Silent Failure*)
- *Rust: s[0] = Compile Error!* ต้องใช้ *.chars()* และ *Panic* ทันที ถ้าตัดขอบเขตผิด (*Fail Fast*)

Trade-off ของ Rust: แม้ Rust จะปลอดภัยกว่า แต่ก็แลกมาด้วย *Learning Curve* ที่สูง (*Steep*) และ *Compile Time* ที่นานขึ้น เนื่องจาก *Compiler* ต้องตรวจสอบกฎ *Ownership* อย่างละเอียด

สรุป: Rust ให้ความสำคัญกับ "*Data Integrity*" มาก่อนความสะดวกของผู้เขียนโค้ด และทั้งหมดนี้เป็นไปได้เพราะมี *Ownership* เป็นรากฐาน

PPL Perspective — มุมมองเชิง Programming Language

Syntax

- Explicit Intent: บังคับ mut และ &
- ห้าม Index String (`s[0]` = Compile Error) เพราะ UTF-8 เป็น Variable-width
- ต้องใช้ `.chars()` เพื่อความปลอดภัยตั้งแต่ตอน Compile

Semantics

- Move Semantics: โอนความเป็นเจ้าของ (ไม่ Copy Reference)
- Fail Fast: Panic ทันทีถ้าตัด UTF-8 ผิดขอบเขต (เช่น `&thai[0..1]`)
- เปลี่ยนจาก Silent Failure เป็น Data Integrity

Type System

- Static, Strong, No Implicit Coercion
- แยก String (Owned) และ `&str` (Borrowed) ชัดจากกัน
- บังคับใช้กฎ Ownership ในระดับ Compiler 100% → ไม่มี Dangling Pointer

Memory / Resource

- RAII (No Garbage Collector)
- Metadata อยู่ Stack / Data อยู่ Heap
- คำนวณ Memory กันที่ Scope (Deterministic) → ไม่มี GC Pause, ไม่มี Leak

Abstraction

- Zero-Cost Abstractions (Traits, Generics, Iterators)
- Compiler ทำ Monomorphization ให้ประสิทธิภาพเท่า Manual Loop ใน C
- ไม่มี Runtime Overhead (ต่างจาก Java/Python)

Why Rust?

- Safety + Performance + Concurrency พร้อมกันในภาษาเดียว
- Fearless Concurrency: Compiler เช็ค Data Race ให้ 100%
- เอาจุดแข็งของ C/C++ (เร็ว) และ Java/Python (ปลอดภัย) มารวมกัน
- ทั้งหมดนี้เป็นไปได้เพราะมี Ownership เป็นรากฐาน

PPL Perspective — มุมมองเชิง Programming Language

Syntax

- Syntax คือ "กฎการเขียนโค้ด" ของภาษา Rust เปรียบเสมือน "ไวยากรณ์ภาษาไทย" ที่บอกว่าเราต้องเรียงคำอย่างไร ประโยคถึงจะถูกต้อง
- จุดเด่นของ Rust คือ "ความชัดเจน" — เราต้องบอกคอมพิวเตอร์ทุกครั้งว่า "ตัวแปรนี้แก้ไขได้" (mut) หรือ "ตัวแปรนี้ยึดมาใช้" (&) ไม่ปล่อยให้เดา และที่สำคัญที่สุด Rust ห้ามใช้ Index กับ String โดยตรง (s[0]) เพราะ Rust รู้ว่า UTF-8 เป็นการเข้ารหัสที่ตัวอักษรแต่ละตัวมีขนาดไม่เท่ากัน (ตัวอักษรไทยกิน 3 bytes, อังกฤษกิน 1 byte) ถ้าเราใช้ Index มันจะไปตัดกลางตัวอักษร ทำให้ข้อมูลพัง

ตัวอย่างโค้ด

let = สร้างตัวแปรใหม่
mut = บอกว่า "ตัวแปรนี้แก้ไขได้"
(ถ้าไม่ใส่ mut จะแก้ไขไม่ได้เลย)

String::from("Hello") = สร้างข้อความ
"Hello" เก็บไว้ในหน่วยความจำ

```
rust
let mut s = String::from("Hello"); // บรรทัดที่ 1
let slice = &s[0..2];              // บรรทัดที่ 2
// let c = s[0];                  // บรรทัดที่ 3
```

& = "ยึด" ข้อมูลจาก s มาดู โดยไม่ก๊อปปี้ทั้งก้อน
[0..2] = ตัดตั้งแต่ Byte ที่ 0 ถึง 2 ได้คำว่า "He"
(ปลอดภัยเพราะระบุขอบเขตชัดเจน)

ถ้าลบคอมเมนต์ออกจะ Error กันที เพราะ Rust ห้ามใช้ Index กับ String โดยตรง
ทำไม? เพราะถ้า s เป็นคำว่า "สวัสดี" ตัวอักษรแรก ("ส") กิน 3 bytes การใช้ s[0] จะไปตัดกลาง ตัวอักษร ทำให้ข้อมูลพัง

วิธีที่ถูกต้อง: ต้องใช้ .chars() เพื่อดึงตัวอักษรจริงๆ ออกมา

PPL Perspective — มุมมองเชิง Programming Language

Semantics

- Semantics คือ "ความหมาย" ของโค้ดว่าเมื่อเขียนแบบนี้แล้วคอมพิวเตอร์จะทำอะไร
- Rust มีความหมายพิเศษที่เรียกว่า "Move Semantics" คือเมื่อเรากำหนดค่า $a = b$ ข้อมูลจะถูก "ย้าย" จาก b ไปให้ a เลย ไม่ใช่การก๊อปปี้ และตัวแปร b จะใช้งานไม่ได้อีกต่อไป
- เปรียบเทียบเหมือนเรามี "ตุ๊กตาหนึ่งตัว" ถ้าเราให้เพื่อนไป (Move) เราก็ไม่มีตุ๊กตาตัวนั้นแล้ว ต่างจากการก๊อปปี้ที่เรายังมีอยู่
- นอกจากนี้ ถ้าเราตัด String ผิดขอบเขต UTF-8 โปรแกรมจะ Panic กันที (หยุดทำงาน) เพื่อป้องกันข้อมูลเสียหาย

ตัวอย่างโค้ด

$s2 = s1$ ข้อมูล "hello" จะถูก Move (ย้าย) จาก $s1$ ไปให้ $s2$
 $s1$ ตอนนี้ว่างเปล่า

ถ้าลบคอมเมนต์จะ Error เพราะ $s1$ ไม่มีข้อมูลแล้ว (ถูกย้ายไปแล้ว)

ถ้าลบคอมเมนต์จะ Panic (Error ตอนรันโปรแกรม) กันที

```
rust
let s1 = String::from("hello"); // บสรทัดที่ 1
let s2 = s1;                     // บสรทัดที่ 2
// println!("{}", s1);          // บสรทัดที่ 3

let thai = String::from("สวัสดี"); // บสรทัดที่ 4
// let slice = &thai[0..1];       // บสรทัดที่ 5
```

ทำไม? คำว่า "สวัสดี" ตัวอักษรแรก ("ส") กินพื้นที่ 3 bytes แต่เราพยายามตัดที่ Byte ที่ 1 (ซึ่งเป็นกลางตัวอักษร)
Rust จึง "หยุดกันที" เพื่อป้องกันข้อมูลที่เพี้ยนไป (Fail Fast)

PPL Perspective — มุมมองเชิง Programming Language

Type System

- Type System คือ "ระบบการแบ่งประเภทข้อมูล" ของภาษา เปรียบเสมือนการแยกกล่องใส่ของ — กล่องใส่ตัวเลข กล่องใส่ข้อความ กล่องใส่ทศนิยม — เพื่อให้คอมพิวเตอร์รู้ว่าข้อมูลแต่ละอย่างใช้งานยังไง
- Rust มี Type System ที่ "เข้มงวดมาก" สังเกตได้จาก:
 1. ไม่แปลง Type ให้อัตโนมัติ — ถ้าฟังก์ชันต้องการ `&str` เราต้องส่ง `&str` เท่านั้น ส่ง `String` ตรงๆ ไม่ได้
 2. แยก `String` กับ `&str` ชัดเจน — `String` คือข้อความที่เราจองหน่วยความจำเอง (Owned) ส่วน `&str` คือข้อความที่ยืมคนอื่นมาดู (Borrowed)
 3. มี Traits — เหมือน "ใบรับรองความสามารถ" ที่บอกว่าข้อมูลชนิดนี้ทำอะไรได้บ้าง

ตัวอย่างโค้ด

ฟังก์ชันนี้บอกว่า "ฉันต้องการรับ `&str` (ข้อความที่ยืมมา) เท่านั้น"

`my_string` มี Type เป็น `String` (ข้อความที่จองหน่วยความจำเอง)

& "ยืม" `my_string` ส่งเข้าไปในฟังก์ชัน
Rust จะแปลง `&String` ให้กลายเป็น `&str` ให้อัตโนมัติ (ปลอดภัย เพราะเป็นการยืม)

ถ้าลบคอมเมนต์จะ Error เพราะเราส่ง `String` (Owned) ตรงๆ ไปให้ฟังก์ชันที่ต้องการ `&str` (Borrowed) เพราะ Type ไม่ตรงกัน! Rust ไม่ยอมให้แปลง Type อัตโนมัติ เราต้องบอกให้ชัดเจนว่าเราอมจ่ายค่าแปลง

```
Rust
fn process(text: &str) { } // unction ที่ 1

let my_string = String::from("Rust"); // unction ที่ 2

process(&my_string); // unction ที่ 3: OK (&String -> &str)

// process(my_string); // unction ที่ 4: Error!
```

PPL Perspective — มุมมองเชิง Programming Language

Memory / Resource

- Memory Management คือ "การจัดการหน่วยความจำ" เปรียบเสมือนการเช่าห้องพัก — เราต้องจองห้อง (จอง Memory) และเมื่อใช้เสร็จต้องคืนห้อง (คืน Memory)
- Rust จัดการ Memory ด้วยหลักการ RAII ที่แปลว่า "เมื่อสร้างขึ้นมา ก็ต้องมีคนรับผิดชอบ และเมื่อหมดหน้าที่ ก็ต้องคืนทันที"
 1. Stack: เก็บข้อมูลขนาดเล็กที่รู้ขนาดแน่นอน (เช่น Pointer ที่ชี้ไปยังข้อมูล)
 2. Heap: เก็บข้อมูลขนาดใหญ่ที่ยืดหยุ่นได้ (เช่น ข้อความ "Hello" ที่อาจยาวขึ้นได้)
- เมื่อตัวแปรหลุดขอบเขตการทำงาน (Scope) หน่วยความจำจะถูกคืนทันที โดยไม่ต้องรอ Garbage Collector ทำให้ไม่มีการโปรแกรมหยุดชั่วคราว (GC Pause)

ตัวอย่างโค้ด

fn create_string() { เริ่มต้นฟังก์ชัน เปิด Scope การทำงาน

let s = String::from("Hello");
จอง Memory บน Heap เพื่อเก็บคำว่า "Hello"
ตัวแปร s เก็บ Pointer ที่ชี้ไปยัง "Hello" ไว้บน Stack

println!("{}", s); ใช้งาน s ได้ตามปกติ

```
fn create_string() { // บรรทัดที่ 1
    let s = String::from("Hello"); // บรรทัดที่ 2
    println!("{}", s); // บรรทัดที่ 3
} // บรรทัดที่ 4
```

} (ปิดกาศปิดฟังก์ชัน) เมื่อฟังก์ชันจบลง ตัวแปร s หลุด Scope
Rust จะเรียกฟังก์ชัน drop() เพื่อคืน Memory บน Heap ทันที
ไม่ต้องรอ Garbage Collector ไม่มี Memory Leak ไม่ต้อง free() เอง

PPL Perspective — มุมมองเชิง Programming Language

Abstraction

- Abstraction คือ "การซ่อนความซับซ้อน" เปรียบเสมือนการขับรถ — เราแค่เหยียบคันเร่ง รถก็วิ่ง โดยที่เราไม่ต้องรู้ว่าข้างในเครื่องยนต์ทำงานยังไง
- Rust มี "Zero-Cost Abstractions" หมายความว่าเราสามารถเขียนโค้ดระดับสูงที่อ่านง่าย (เช่น การใช้ map, filter กับ Collections) แต่ Compiler จะแปลงให้กลายเป็นโค้ดที่เร็วเท่ากับการเขียน Loop ธรรมดาในภาษา C เลย ไม่มีค่าใช้จ่ายแอบแฝง
- และ Abstraction ยังช่วยซ่อนความซับซ้อนของ UTF-8 เช่น การใช้ .chars() เพื่อวนลูปอ่านตัวอักษรจริงๆ โดยที่เราไม่ต้องคำนวณ Byte เอง

ตัวอย่างโค้ด

สร้าง Collection (Vec) ที่เก็บคำ 3 คำ

สร้าง Iterator เพื่อวนอ่านข้อมูลใน words ทีละคำ

แปลงแต่ละคำให้กลายเป็น "ความยาว" (ความยาวของ "Rust" = 4, "is" = 2, "fast" = 4)

.sum() = รวมความยาวทั้งหมด (4 + 2 + 4 = 10)

```
Rust

let words = vec!["Rust", "is", "fast"]; // บรรทัดที่ 1

let total_len: usize = words
    .iter()                // บรรทัดที่ 2.1
    .map(|w| w.len())      // บรรทัดที่ 2.2
    .sum();                // บรรทัดที่ 2.3
```

โค้ดนี้ดูเหมือนทำงานซับซ้อน (มีทั้ง iter, map, sum) แต่ Compiler จะแปลงให้กลายเป็น Loop ธรรมดาที่เร็วมาก ไม่มี Overhead จากการเรียกฟังก์ชันซ้อนกัน

PPL Perspective — มุมมองเชิง Programming Language

Why Rust?

- คำถามใหญ่ของวงการซอฟต์แวร์คือ: "ทำไมเราไม่มีภาษาที่ได้ทั้งความเร็วและความปลอดภัยไปพร้อมกัน?"
- ภาษา C/C++ เร็วมาก แต่ต้องจัดการ Memory เอง เสี่ยงโปรแกรมพัง
- ภาษา Java/Python ปลอดภัยกว่า แต่ช้ากว่าเพราะมี Garbage Collector มาคอยจัดการ
- Rust ตอบโจทย์นี้ ด้วยระบบ Ownership ทำให้ Compiler ตรวจสอบความปลอดภัยให้เราตั้งแต่ตอน Compile ถ้าโค้ดไม่ปลอดภัย มันจะไม่ยอมให้ Compile ผ่านเลย ทำให้เราได้ทั้ง ความเร็วระดับ C และ ความปลอดภัยระดับ Java โดยไม่มี GC

ตัวอย่างโค้ด

use std::thread; เรียกใช้โมดูลสำหรับสร้าง Thread (การทำงานพร้อมกัน)

let v = vec![1, 2, 3]; สร้าง Collection เก็บตัวเลข 1, 2, 3

thread::spawn(move || {
thread::spawn = สร้าง Thread ใหม่ (ทำงานพร้อมกัน)
move = สำคัญที่สุด! บอกว่าให้ย้าย Ownership
ของ v ไปให้ Thread ใหม่

println!("{:?}", v);
Thread ใหม่พิมพ์ค่า v ออกมาได้อย่างปลอดภัย 100%
ทำไมปลอดภัย? เพราะ v ถูก Move ไปให้ Thread ใหม่แล้ว
Thread หลักไม่มี v อีกต่อไป
ถ้าไม่มี move Compiler จะ Error กันที เพราะนี่คือ Data
Race (ข้อมูลถูกแก้ไขพร้อมกัน)

```
Rust
use std::thread; // บรรทัดที่ 1

let v = vec![1, 2, 3]; // บรรทัดที่ 2

thread::spawn(move || { // บรรทัดที่ 3
    println!("{:?}", v); // บรรทัดที่ 4
}); // บรรทัดที่ 5
```

}); จบการสร้าง Thread

Common Mistakes — ข้อผิดพลาดที่พบบ่อย

✗ Incorrect

```
// [โค้ดที่ผิด / ตัวอย่างข้อผิดพลาด]
fn print_message(msg: String) {
    println!("message {}", msg);
}

fn main() {
    let my_str = String::from("Hello Rust");
    print_message(my_str);

    println!("print again: {}", my_str);
}
```

✓ Correct

```
// [โค้ดที่แก้ไขถูกต้อง]
fn print_message(msg: &String) {
    println!("message: {}", msg);
}

fn main() {
    let my_str = String::from("Hello Rust");
    print_message(&my_str);

    println!("print again: {}", my_str);
}
```

ทำไมถึงผิด? (Why?)

การที่คุณส่งข้อมูล string หรือ vec เข้าไปในฟังก์ชันโดยไม่ใช้ "&" จะทำให้สิทธิ์ความเป็นเจ้าของถูกย้ายไปเพราะไม่ได้ใช้ "&" ทำให้ไม่สามารถใช้ตัวแปรเดิมได้อีก

Common Mistakes — ข้อผิดพลาดที่พบบ่อย

✗ Incorrect

```
// [โค้ดที่ผิด / ตัวอย่างข้อผิดพลาด]
fn main() {
    let mut my_vec = vec![1, 2, 3];
    for item in &my_vec {
        if *item == 2 {
            my_vec.push(4);
        }
    }
}
```

✓ Correct

```
// [โค้ดที่แก้ไขถูกต้อง]
fn main() {
    let mut my_vec = vec![1, 2, 3];
    let mut to_add = Vec::new();
    for item in &my_vec {
        if *item == 2 {
            to_add.push(4);
        }
    }
    my_vec.append(&mut to_add);
    println!("{:?}", my_vec);
}
```

ทำไมถึงผิด? (Why?)

การพยายามเพิ่ม/ลบ ข้อมูลใน Collection ขณะกำลังวนลูปอ่านในภาษา Rust จะไม่อนุญาตเพราะ การเพิ่มข้อมูลอาจทำให้ Vector ต้องย้ายที่อยู่บน Heap ซึ่งจะทำให้ตัวแปรใน Loop ชี้ไปที่หน่วยความจำที่พังไปแล้ว

Challenge — คำถามท้าทายผู้ฟัง

โจทย์ (Problem)

คุณได้รับข้อมูลจากการแข่งขันเกม ซึ่งมาในรูปแบบข้อความยาวๆ เพียงบรรทัดเดียว (&str) โดยข้อมูลจะอยู่ในรูปแบบ "ชื่อ:คะแนน,ชื่อ:คะแนน" ติดกันไปเรื่อยๆ เช่น let raw_data = "Somchai:75,Somsri:95,Mana:82,Piti:60";

ให้เขียนฟังก์ชัน find_champion ที่รับค่า raw_data เข้าไป (ชนิดเป็น &str) แล้วให้ฟังก์ชันนี้ส่งผลลัพธ์กลับมาเป็น Tuple ที่หน้าตาแบบนี้ (String, u32, Vec<String>)

โดยสิ่งที่อยู่ใน Tuple 3 ช่องนี้คือ

- ช่องที่ 1 (String): ชื่อของคนที่ได้คะแนน "สูงสุด"
- ช่องที่ 2 (u32): คะแนนที่สูงที่สุดนั้น
- ช่องที่ 3 (Vec<String>): รายชื่อของ "คนอื่นๆ ที่เหลือ" (เรียงลำดับยังงี้ก็ได้)



Hint

1. ฟังก์ชันคุณควรจะหน้าตาแบบนี้: fn find_champion(data: &str) -> (String, u32, Vec<String>)
2. ใช้ data.split(',') เพื่อแยกผู้เล่นแต่ละคนออกมาก่อน แล้วค่อยใช้ .split_once(':') เพื่อหั่นชื่อกับคะแนนออกจากกัน
3. ข้อมูลที่หั่นได้ยังมีชนิดเป็น &str ต้องใช้ .to_string() แปลงชื่อคนให้เป็น String ก่อนเก็บลง Tuple หรือ Vector
4. ถ้าเจอแชมป์คนใหม่ อย่าลืมเอาชื่อแชมป์คนเก่าเข้าไปอยู่ในกลุ่ม vec ของคนแพ้

```
fn find_champion(data: &str) -> (String, u32,
Vec<String>) {
    let mut champ_name = String::new();
    let mut max_score = 0;
    let mut runner_ups: Vec<String> =
Vec::new();
```

Challenge — คำถามท้าทายผู้ฟัง

```
for item in data.split(',') {  
  
    if let Some((name, score_str)) =  
item.split_once(':') {  
  
        let score: u32 =  
score_str.parse().unwrap_or(0);  
  
        if score > max_score {  
            if !champ_name.is_empty() {  
                runner_ups.push(champ_name);  
            }  
        }  
    }  
}
```

```
        champ_name = name.to_string();  
        max_score = score;  
  
    } else {  
  
runner_ups.push(name.to_string());  
    }  
}  
  
(champ_name, max_score, runner_ups)  
}
```

Challenge — คำถามท้าทายผู้ฟัง

```
fn main() {  
    let raw_data =  
    "Somchai:75,Somsri:95,Mana:82,Piti:60";  
  
    let (winner, score, others) =  
    find_champion(raw_data);  
  
    println!("ผู้ชนะคือ: {} ({} คะแนน)", winner,  
score);  
    println!("ผู้เข้าแข่งขันท่านอื่นๆ: {:?}", others);  
}
```

ผลลัพธ์

```
ผู้ชนะคือ: Somsri (95 คะแนน)  
ผู้เข้าแข่งขันท่านอื่นๆ: ["Somchai", "Mana", "Piti"]
```

Key Takeaways — สรุปสิ่งที่ควรรจำ

- 1 [Collection ใช้จัดเก็บข้อมูลหลายค่าให้อยู่เป็นกลุ่ม เช่น Array เก็บข้อมูลจำนวนคงที่, Tuple เก็บข้อมูลต่างชนิดกันได้, Vector สามารถเพิ่ม-ลดจำนวนข้อมูลได้, String ใช้จัดการข้อความที่เปลี่ยนแปลงได้ และ &str ใช้อ้างอิงข้อความที่มีอยู่แล้ว]
- 2 [String คือเจ้าของข้อมูลที่ยึดหดได้ (Heap) ส่วน &str คือการยืมข้อความมาอ่านชั่วคราว (Slice) หากฟังก์ชันมีหน้าที่แค่ "อ่าน" ข้อความควรตั้งพารามิเตอร์เป็น &str]
- 3 [String ใน Rust ชับซ้อนกว่าภาษาอื่นเพราะเข้ารหัสแบบ UTF-8 (ตัวอักษรหนึ่งตัวอาจมีขนาด 1-4 ไบต์)]
- 4 [Collection & String รวมเอากลไกและแนวคิดหลักเกือบทุกหัวข้อของภาษา Rust มาทำงานร่วมกัน เช่น Ownership, References & Borrowing, Function]

References — เอกสารอ้างอิง

- The Rust Programming Language (The Rust Book) — <https://doc.rust-lang.org/book/>
- Rust by Example — <https://doc.rust-lang.org/rust-by-example/>
- [Official documentation ที่เกี่ยวข้องกับหัวข้อนี้] - <https://doc.rust-lang.org/book/ch08-00-common-collections.html>
- [แหล่งอ้างอิงเพิ่มเติม] - <https://doc.rust-lang.org/rust-by-example/primitives/array.html>

GitHub Repository: <https://github.com/670710146/collection-string>

Questions & Answers

กลุ่ม [10] — [Collection & String]